

Beginner-C Bootcamp 4주차 과제

20215789 산업보안학과 김문정

Challenge 1. 함수 작성 및 호출 구조 이해

먼저 시저 암호가 무엇인지 몰라 찾아보니, 어떤 문장의 각 알파벳을 일정한 거리만큼 밀어서 다른 알파벳으로 바꾸는 암호화 방식이라는 것을 알 수 있었다. 예를 들어 "AB"는 1만큼 밀면 "BC"가 되고, 3만큼 밀면 "DE"가 된다. "z"는 1만큼 밀면 "a"가 되는 순환 방식의 암호인 것이다. 시프트 라는 단어가 나와서 지난 챌린지에서 학습한 $>>$, $<<$ 를 써야하는건지 생각하였지만, 그것이 아니고 각 알파벳에 해당하는 아스키 값을 이용해 코드를 작성해야 한다는 것을 알 수 있었다.

```
void encode(char *str, int shift) {  
  
    for(int i = 0; str[i] != '\0'; i++) {  
  
        str[i] = str[i] + shift;  
  
    }  
  
}
```

여기서 입력된 배열은 문자 배열이고, `str[i]`도 실제로 문자를 저장하는데 문자와 숫자 연산이 어떻게 가능한지 궁금하였다. 찾아보니 내부적으로 문자가 아스키 코드값으로 저장되기 때문에 정수인 `shift`값과 연산이 가능하다는 것을 알 수 있었다.

10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자
64	0x40	@	80	0x50	P	96	0x60	`	112	0x70	p
65	0x41	A	81	0x51	Q	97	0x61	a	113	0x71	q
66	0x42	B	82	0x52	R	98	0x62	b	114	0x72	r
67	0x43	C	83	0x53	S	99	0x63	c	115	0x73	s
68	0x44	D	84	0x54	T	100	0x64	d	116	0x74	t
69	0x45	E	85	0x55	U	101	0x65	e	117	0x75	u
70	0x46	F	86	0x56	V	102	0x66	f	118	0x76	v
71	0x47	G	87	0x57	W	103	0x67	g	119	0x77	w
72	0x48	H	88	0x58	X	104	0x68	h	120	0x78	x
73	0x49	I	89	0x59	Y	105	0x69	i	121	0x79	y
74	0x4A	J	90	0x5A	Z	106	0x6A	j	122	0x7A	z
75	0x4B	K	91	0x5B	[	107	0x6B	k	123	0x7B	{

각 문자에 해당하는 아스키 값은 다음과 같다. 한편 위에 적은 코드에는 문제점이 있었는데, 위 코드는 단순히 아스키값을 더하기 때문에 알파벳 마지막 소문자인 z에서 1을 더할 경우 a로 돌아가지 않고 다음 아스키값에 해당하는 [을 출력한다는 것을 알 수 있었다. 따라서, 알파벳 순환구조가 되도록 $\%26$ 을 해주어야 한다.

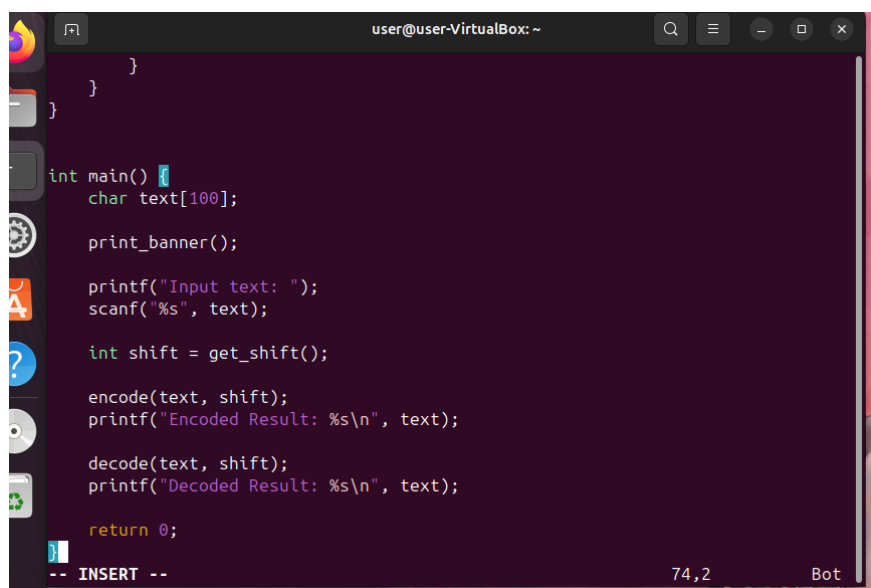
또한 알파벳 대문자와 소문자를 구분해줘야 하며, 따라서 최종 encode함수는 다음과 같다.

```
for (int i = 0; str[i] != '\0'; i++) {  
  
    char c = str[i];  
  
    if (c >= 'A' && c <= 'Z') {  
  
        str[i] = (c - 'A' + shift) % 26 + 'A';  
  
    }  
  
    else if (c >= 'a' && c <= 'z') {  
  
        str[i] = (c - 'a' + shift) % 26 + 'a';  
  
    }  
}
```

코드를 설명하면, 끝에 널문자를 만날때까지 입력을 받고, c에 문자열을 저장한다. 먼저, c-'A'를 하여 알파벳을 숫자로 바꾼다. 또한, 거기에 사용자가 입력한 이동하고 싶은 shift를 더하고, 알파벳이 26개이므로 범위를 넘어가는 숫자를 다시 처음으로 돌리기 위해 %26을 한다. 마지막으로, 이것을 다시 문자로 변환시키기 위해 A를 더해준다. 소문자 일때도 마찬가지로 방법이다.

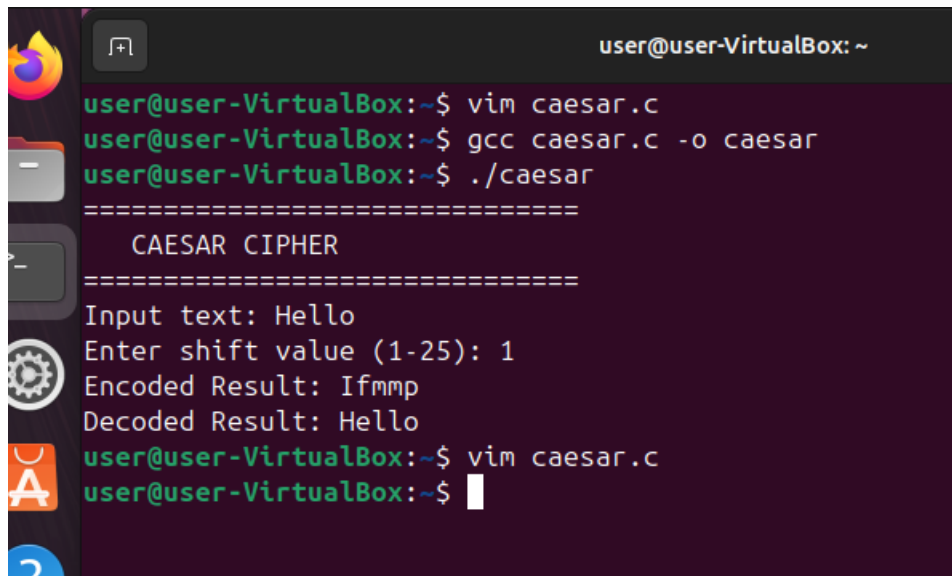
복호화할 때는, 암호화에서 +shift를 했으므로 -shift를 해주고, (c - 'A' - shift)가 음수가 될 수 있으므로 +26을 더해준 코드인 str[i] = (c - 'A' - shift + 26) % 26 + 'A'; 를 쓰면 된다.

main함수 코드는 다음과 같다.

A screenshot of a terminal window titled 'user@user-VirtualBox: ~'. The terminal displays C code for a Caesar cipher program. The code includes a main function that calls print_banner(), prompts for input text, reads it with scanf, gets a shift value from get_shift(), and then calls encode and decode functions. The encode function uses the logic discussed in the text: (c - 'A' + shift) % 26 + 'A' for uppercase and (c - 'a' + shift) % 26 + 'a' for lowercase. The decode function is not fully visible but is mentioned in the text. The terminal shows the code being typed, with some lines already present. The bottom of the terminal shows a cursor at line 74, column 2, and a 'Bot' status.

```
    }  
}  
  
int main() {  
    char text[100];  
  
    print_banner();  
  
    printf("Input text: ");  
    scanf("%s", text);  
  
    int shift = get_shift();  
  
    encode(text, shift);  
    printf("Encoded Result: %s\n", text);  
  
    decode(text, shift);  
    printf("Decoded Result: %s\n", text);  
  
    return 0;  
}  
-- INSERT --  
74,2 Bot
```

또한 우선 잘 작동하는지 확인하기 위해 임시로 컴파일한 후 결과를 확인해보면

A terminal window titled 'user@user-VirtualBox: ~' showing the execution of a C program. The user runs 'vim caesar.c', 'gcc caesar.c -o caesar', and './caesar'. The program outputs a banner 'CAESAR CIPHER' and prompts for input text and shift value. The user enters 'Hello' and '1', resulting in 'Encoded Result: Ifmmp' and 'Decoded Result: Hello'. The user then runs 'vim caesar.c' again.

```
user@user-VirtualBox:~$ vim caesar.c
user@user-VirtualBox:~$ gcc caesar.c -o caesar
user@user-VirtualBox:~$ ./caesar
=====
CAESAR CIPHER
=====
Input text: Hello
Enter shift value (1-25): 1
Encoded Result: Ifmmp
Decoded Result: Hello
user@user-VirtualBox:~$ vim caesar.c
user@user-VirtualBox:~$
```

다음과 같이 잘 작동하는 모습을 볼 수 있다.

코드가 길어 전체 코드를 복사 붙여넣기 하기에는 어렵지만, 간단히 요약하자면 우선 챌린지에서 정의한대로 main함수 진입하기 전, 배너를 출력해주는 print_banner, 사용자의 shift값을 입력받는 get_shift함수를 작성했다. 또한, 문자열을 shift만큼 밀고 이를 다시 복호화해주는 encode함수와 decode함수도 작성했다. 이후 main함수로 넘어가, print_banner함수를 부르고, 사용자가 텍스트와 shift값을 입력하도록 하였다. 이후 encode, decode 함수를 부르고 인자에 사용자가 입력한 text와 shift가 넘어가도록 하였다.

이를 통해 본 함수의 유용성은

1. 코드의 재사용성이다. main함수 진입 전 함수를 정의하여 필요시 어디서든 코드를 다시 쓰지 않고 함수를 호출하여 사용할 수 있다.
2. 코드의 가독성 향상이다. main함수 안에 모든 코드를 넣으면 읽기 힘든데, 함수를 쓰면 기능별로 코드를 나누어 쓸 수 있어 가독성이 향상된다.
3. 유지보수의 용이성이다. 문제가 생기면 모든 함수를 처음부터 끝까지 보면서 수정하는 것이 아니라, 해당 함수의 내부만 수정하면 되므로 유지보수가 쉽다.

따라서 함수를 사용하는 이유는 위 3가지 유용성을 통해 프로그램을 보다 효율적이고 체계적으로 작성하기 위해서라는 것을 학습할 수 있었다.

Challenge 2. gdb로 스택 프레임 관찰

참고자료: https://velog.io/@got1_c/Stack

https://velog.io/@got1_c/Stack-Frame

스택이란 임시데이터를 저장하거나, 함수 호출 시 복귀 주소와 인자, 지역 변수를 관리하는데 사용되는 후입선출 자료구조이다.

스택 포인터(SP, Stack Pointer)

스택 포인터는 **top**을 가리키는 주소 레지스터를 말한다.

--> 마지막으로 데이터가 추가된 곳을 가리키는 레지스터

SP가 하는 역할

SP는 top을 가리키는 주소 레지스터이므로, CPU가 스택에 접근할때 항상 이 값을 기준으로 한다.

즉, **top**이 변하면 **SP**도 변한다.

push: SP가 감소하고 새 데이터가 [SP]에 저장된다.

pop: [SP]의 데이터를 꺼내고 [SP]가 증가한다.

이를 바탕으로 한 스택 프레임은 함수가 호출될 때 스택 내에 새롭게 형성되는 독립적인 영역이다. 함수가 실행되는 동안 필요한 매개변수(Arguments), 함수 실행 후 돌아갈 복귀 주소(Return Address), 그리고 함수 내부에서 선언된 지역 변수(Local Variables) 등이 저장된다.

스택 프레임이 필요한 이유

프로그램은 한 번에 여러 개의 함수를 호출할 수 있고, 재귀 호출도 가능하다.

이때, 각 함수의 매개변수, 복귀 주소, 지역 변수가 섞이지 않도록 함수마다 독립적인 저장 공간이 필요하다. -> 그 공간이 바로 스택 프레임

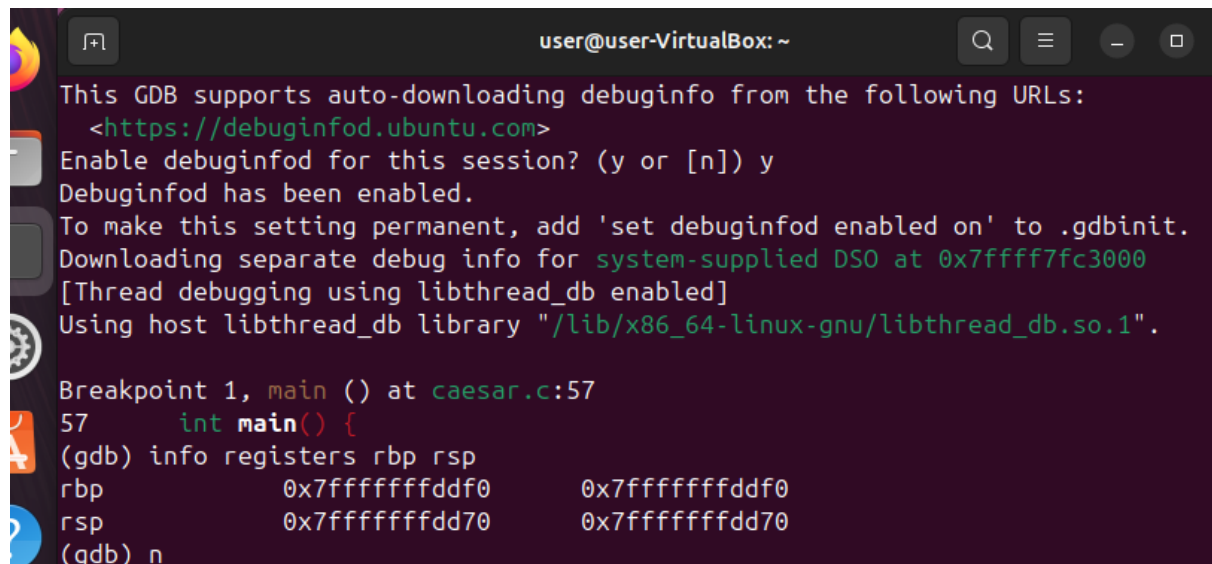
함수가 시작될 때 생성되고, 함수가 종료되면 사라진다.

우선 rbp, rsp란

rbp (Base Pointer): 현재 스택 프레임의 기준점(바닥)

rsp (Stack Pointer): 현재 스택의 최상단(끝)

을 말한다.



```
user@user-VirtualBox: ~  
This GDB supports auto-downloading debuginfo from the following URLs:  
<https://debuginfod.ubuntu.com>  
Enable debuginfod for this session? (y or [n]) y  
Debuginfod has been enabled.  
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.  
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000  
[Thread debugging using libthread_db enabled]  
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".  
  
Breakpoint 1, main () at caesar.c:57  
57      int main() {  
(gdb) info registers rbp rsp  
rbp      0x7fffffffddfd0      0x7fffffffddfd0  
rsp      0x7fffffffdd70      0x7fffffffdd70  
(gdb) n
```

main함수에 들어가 rbp, rsp값을 확인하였다. (함수 호출 전) 이제 함수를 호출하고 그에 따른 rbp, rsp값을 확인하기 위해 encode함수가 나올때까지 n(next)를 누른다. encode가 보이면 step을 눌러 함수 안으로 들어간다.

encode (str=0x7fffffffdd80 "n", shift=32767) at caesar.c:21

21 shift %= 26; // 다음에 실행할 명령어

가 의미하는 바는

str=0x7fffffffdd80 "n": main 함수에 있던 문자열(text)의 메모리 주소이고, 현재 그 주소에 "n"이라는 글자가 들어있다는 뜻이다.

shift=32767: ⚠ 숫자가 너무 큰데 이유는 함수의 첫번째 줄 실행 전이라 메모리에 shift 변수 공간만 잡혀 있고 실제 값 저장 전의 쓰레기값이 보이고 있는 것이다.

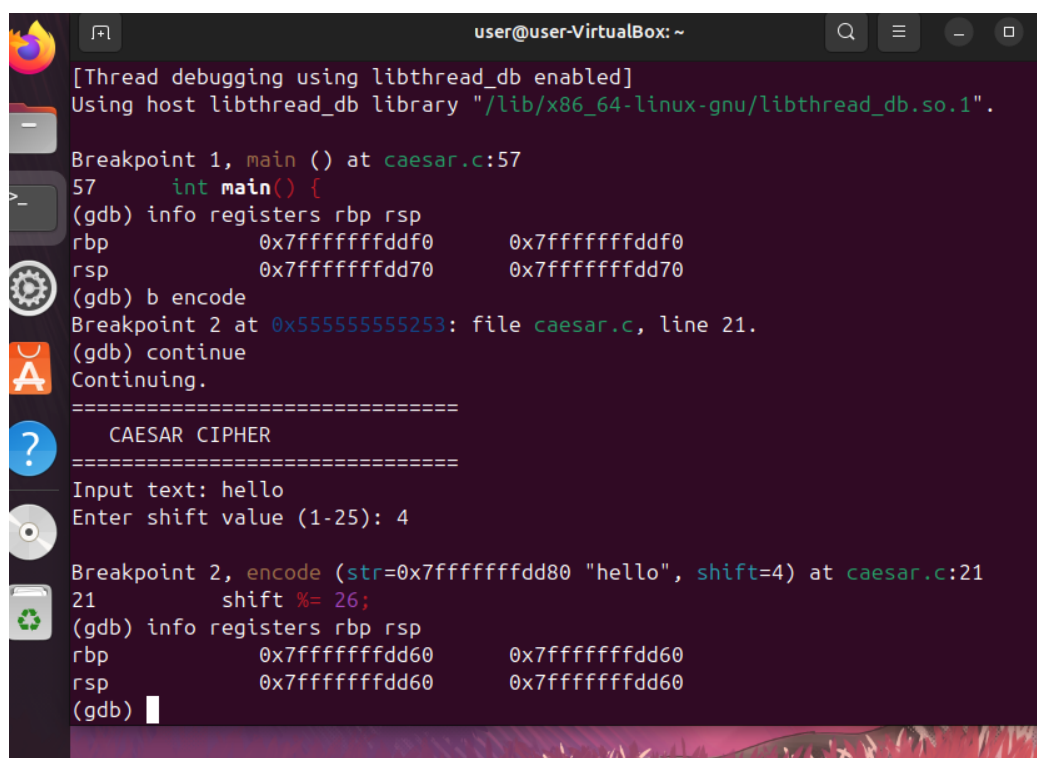
---다시 시작---

main함수에 브레이크 포인트를 걸고 info registers rbp rsp를 확인한다. (함수 호출 전)

main함수의 상태는

- RBP (바닥): 0x7fffffffddfd0
- RSP (꼭대기): 0x7fffffffdd70

이후 encode에 다시 브레이크포인트를 걸어주고 info registers rbp rsp를 확인한다.



```
user@user-VirtualBox: ~  
[Thread debugging using libthread_db enabled]  
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".  
Breakpoint 1, main () at caesar.c:57  
57 int main() {  
(gdb) info registers rbp rsp  
rbp             0x7fffffffddfd0      0x7fffffffddfd0  
rsp             0x7fffffffdd70      0x7fffffffdd70  
(gdb) b encode  
Breakpoint 2 at 0x55555555253: file caesar.c, line 21.  
(gdb) continue  
Continuing.  
===== CAESAR CIPHER =====  
Input text: hello  
Enter shift value (1-25): 4  
Breakpoint 2, encode (str=0x7fffffffdd80 "hello", shift=4) at caesar.c:21  
21 shift %= 26;  
(gdb) info registers rbp rsp  
rbp             0x7fffffffdd60      0x7fffffffdd60  
rsp             0x7fffffffdd60      0x7fffffffdd60  
(gdb)
```

encode 함수 호출 후 rbp, rsp가 변한 것을 확인할 수 있다.

main함수의 rsp가 dd70이었는데, encode의 함수에서는 rsp가 dd60으로 내려간 것을 확인할 수 있다. 이는 함수가 호출되었기 때문에, 값들이 push되어서 rsp가 작아졌기 때문이다.

encode 함수를 진입하게 되면

```
push rbp          // 이전 rbp 저장 - 이전 함수로 돌아가기 위해 필요  
mov rbp, rsp      // 현재 rsp를 rbp로 설정
```

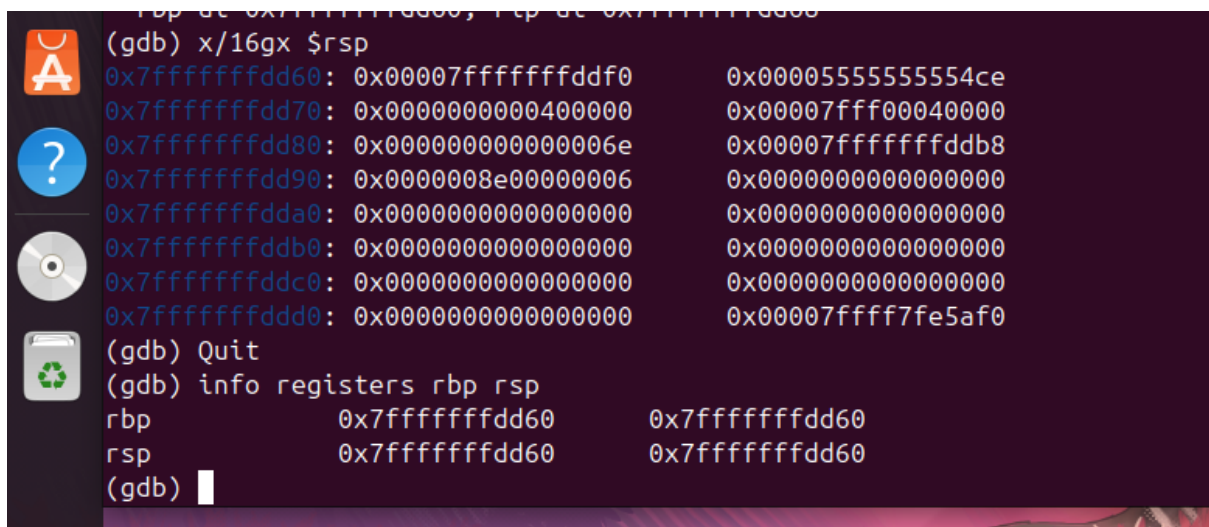
따라서 rbp = rsp = 0x7fffffffdd60

정리하자면

rsp: 스택의 맨 꼭대기

rbp: 함수 시작 기준 좌표, encode 끝나면 main으로 돌아가야 하니까 main의 rbp 복구
함수가 호출되면 rsp는 낮아지고, 그 위치에서 rbp가 설정된다.

x/16gx \$rsp를 쳐보면 다음과 같이 나온다.



```
(gdb) x/16gx $rsp
0x7fffffffdd60: 0x00007fffffffdddf0      0x00005555555554ce
0x7fffffffdd70: 0x00000000000040000      0x00007fff00040000
0x7fffffffdd80: 0x0000000000000006e      0x00007fffffffddb8
0x7fffffffdd90: 0x00000008e00000006      0x0000000000000000
0x7fffffffdda0: 0x00000000000000000      0x0000000000000000
0x7fffffffddb0: 0x00000000000000000      0x0000000000000000
0x7fffffffddc0: 0x00000000000000000      0x0000000000000000
0x7fffffffddd0: 0x00000000000000000      0x00007ffff7fe5af0
(gdb) Quit
(gdb) info registers rbp rsp
rbp                0x7fffffffdd60      0x7fffffffdd60
rsp                0x7fffffffdd60      0x7fffffffdd60
(gdb) 
```

encode함수가 끝난 뒤에는 다시 main함수로 돌아와야 한다. 이때 main()함수의 어느곳
으로 돌아와야 하는지 위치를 정해둔 것이 Ret address이다. 일단 encode가 호출되면,
메인 함수로 돌아가서 실행할 다음 줄의 주소를 스택에 push(ret address)하고, main으로
도 돌아가야 하니 main의 rbp또한 적어둔다. 그것이 첫 번째 0x7fffffffdd60:
0x00007fffffffddf0 0x00005555555554ce 의미인 것을 알 수 있었다.

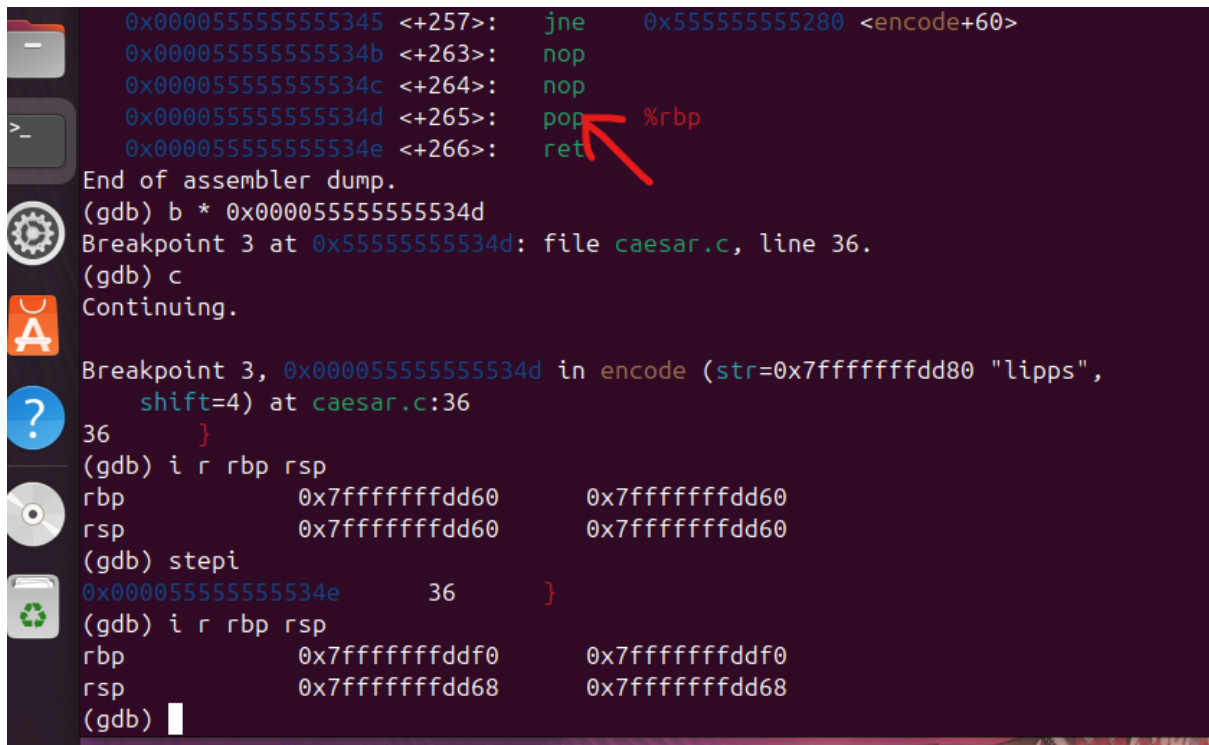
0x7fffffffdd60: 현재 스택의 rsp

0x00007fffffffddf0: main 함수의 rbp

0x00005555555554ce: Ret Address

leave가 실행되면, encode함수가 시작될 때 만들어두었던 스택 프레임을 없애고 다시 main으로 돌아간다. (rsp를 rbp로 옮겨서 encode를 날리고, 스택에 저장해둔 main함수의 rbp값을 꺼내어 다시 rbp에 넣는다.) 또한 encode함수를 호출했던 곳으로 돌아가야 하니, ret 명령어가 스택에 남은 Ret address를 꺼내어 그 주소로 점프하게 된다.

gdb를 다시 실행시킨다. encode에 브레이크 포인트를 걸고, 프로그램을 실행시킨다. leave가 실행되는 주소를 보기 위해, disas encode를 치고 아래로 내려본다.



```
0x000055555555345 <+257>: jne    0x555555555280 <encode+60>
0x00005555555534b <+263>: nop
0x00005555555534c <+264>: nop
0x00005555555534d <+265>: pop    %rbp
0x00005555555534e <+266>: ret

End of assembler dump.
(gdb) b * 0x00005555555534d
Breakpoint 3 at 0x5555555534d: file caesar.c, line 36.
(gdb) c
Continuing.

Breakpoint 3, 0x00005555555534d in encode (str=0x7fffffffdd80 "lipps",
shift=4) at caesar.c:36
36 }
(gdb) i r rbp rsp
rbp      0x7fffffffdd60      0x7fffffffdd60
rsp      0x7fffffffdd60      0x7fffffffdd60
(gdb) stepi
0x00005555555534e      36      }
(gdb) i r rbp rsp
rbp      0x7fffffffddf0      0x7fffffffddf0
rsp      0x7fffffffdd68      0x7fffffffdd68
(gdb)
```

화면과 같이 0x00005555555534d 에서 leave가 실행되는 것을 알 수 있다. 따라서 저 주소에 b를 걸고, 이전과 이후의 rbp rsp값을 확인해보면 된다.

leave가 실행되고 나면, rbp는 main함수의 rbp로 바뀐 것을 확인할 수 있고, rsp는 위에서 본 ret address가 저장되어 있는 주소로 바뀌게 되어 dd68로 됨을 확인할 수 있다.